Assignment-7

* Problem Statement : Implement stack on an ADT & apply it for different expression.
Conversions - Infix to Postfix
Prefix to Postfix
Prefix to Infix
Postfix to Infix
Postfix to Prefix

* Objective : i) To study stack & its operation

ii) To study the importance of expression conversion

* Test Condition :

Input	Conversion	Output
1. A+B*C	Infix -> Postfix	ABC*+
2. (A+B)*C	Infix -> Postfix	AB+C*
3. ABC*+	Postfix -> Infix	A+(B*C)
4. AB+C*	Postfix -> Infix	(A+B)*C

* Psuedo Code

Push:
    If stack is full → Overflow
    Else Top ++
    Stack[Top] = Item

Pop:
    If stack is empty → Underflow
    Else Item = Stack[Top]
    Top --

Infix to PostFix:
    Scan expression left to right
    Operand → Add to output
    '(' → Push
    ')' → Pop until '('
    operator → Pop higher/equal precedence
    operator then push
    Pop remaining operator

PostFix to Infix:
    Scan expression left to right
    operand → Push
    Operator → Pop two operands
    Combine As Op1 operator Op2
    Push Result
    Final stack element = Infix Expression.

- Time Complexity :

Operation	Complexity
Push	O(1)
Pop	O(1)
Infix -> Postfix	O(n)
Postfix -> Infix	O(n)

* FAQs

Q1 What is the advantage of PreFix & Postfix over infix expression.

A- Prefix & Postfix expression do not require paranth

- They do not need operator precedence rules dur evaluation.
- They are faster to & easy for computers evaluate prefix & postfix using stack than infix.

Q2 Why is stack used in Expression conversion?

A- The stack follows LIFO. It is useful for temporary storage of operators & operands.

- It helps to handle its correct order during infix, prefix & postfix conversion
- It helps the computer to avoid the need of backtracking multiple times to find higher precedence.

Q3. Give Stack full & Stack Empty conditions for stack.

- Stack full: When Top = Max-1, no more elements can be inserted.

  • If (Top = Max-1)
  → Stack Overflow

A

- Stack Empty: When Top = -1, it means that there are no elements in the Stack.

  • If (Top = -1)
  → Stack Underflow.

PRACTICAL 7

EXPRESSION CONVERSION USING STACK

CSF20030 – Data Structures Laboratory

Experiment 07: Infix to Postfix Conversion

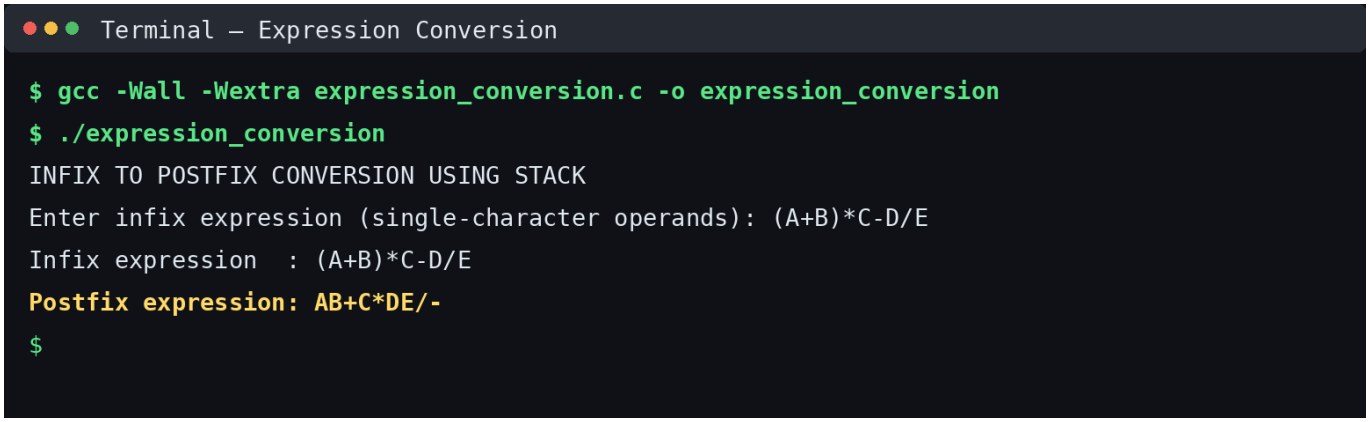
Aim: Implement a stack and use it to convert an infix expression to postfix notation.

C Program

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>
#define MAX 200
char stack[MAX];
int top = -1;
void push(char ch) {
    if (top == MAX - 1) {
        printf("Error: Stack overflow.\n");
        exit(EXIT_FAILURE);
    }
    stack[++top] = ch;
}
char pop(void) {
    if (top == -1) {
        printf("Error: Stack underflow.\n");
        exit(EXIT_FAILURE);
    }
    return stack[top--];
}
char peek(void) {
    if (top == -1) return '\0';
    return stack[top];
}
int precedence(char op) {
    switch (op) {
        case '^': return 3;
        case '*':
        case '/':
        case '%': return 2;
        case '+':
        case '-': return 1;
        default: return 0;
    }
}
/* Convert an infix expression to postfix.
   Operands should be single letters/digits; spaces are ignored. */
void infixToPostfix(const char infix[], char postfix[]) {
    int j = 0;
    top = -1;
    for (int i = 0; infix[i] != '\0'; i++) {
        char ch = infix[i];
        if (isspace((unsigned char)ch)) continue;
        if (isalnum((unsigned char)ch)) {
            postfix[j++] = ch;
        } else if (ch == '(') {
            push(ch);
        } else if (ch == ')') {
            while (top != -1 && peek() != '(')
                postfix[j++] = pop();
            if (top == -1) {
                printf("Error: Mismatched parentheses.\n");
                exit(EXIT_FAILURE);
            }
            pop(); /* discard '(' */
        } else if (strchr("+-*/%^", ch) != NULL) {
            while (top != -1 && peek() != '(' &&
                   (precedence(peek()) > precedence(ch) ||
                   (precedence(peek()) == precedence(ch) && ch != '^'))) {
                postfix[j++] = pop();
            }
            push(ch);
        } else {
            printf("Error: Invalid character '%c'.\n", ch);
            exit(EXIT_FAILURE);
        }
    }
    while (top != -1) {
        char op = pop();
        if (op == '(') {
            printf("Error: Mismatched parentheses.\n");
            exit(EXIT_FAILURE);
        }
        postfix[j++] = op;
```

```
    }
    postfix[j] = '\0';
}
int main(void) {
    char infix[MAX], postfix[MAX];
    printf("INFIX TO POSTFIX CONVERSION USING STACK\n");
    printf("Enter infix expression (single-character operands): ");
    if (fgets(infix, sizeof(infix), stdin) == NULL) {
        printf("Input error.\n");
        return 1;
    }
    infixToPostfix(infix, postfix);
    printf("Infix expression  : %s", infix);
    printf("Postfix expression: %s\n", postfix);
    return 0;
}
```

Sample Input and Output Screenshot



Input: (A+B)*C-D/E
Output: AB+C*DE/-
The screenshot illustrates the compiled program converting the infix expression into postfix form.